

"Sending a message is easy — building a reliable distributed workflow is harder."

MODULE 4

Microservices Integration

Connecting independent services through messaging and events.

**Asynchronous
communication**

**Messaging
fundamentals**

Message brokers

**Event-driven
architecture**

**Reliable message
processing**

**Integration
pattern selection**

LEARNING OBJECTIVES

Messaging concepts, brokers, events, reliability and integration decisions.



BY THE END OF THIS MODULE, YOU CAN

1. Explain asynchronous messaging
2. Describe message-broker architecture
3. Compare queues and pub/sub
4. Explain event-driven architecture
5. Distinguish commands and events
6. Compare choreography and orchestration
7. Design reliable consumers
8. Handle retries and failures
9. Select appropriate integration patterns



THREE PARTS

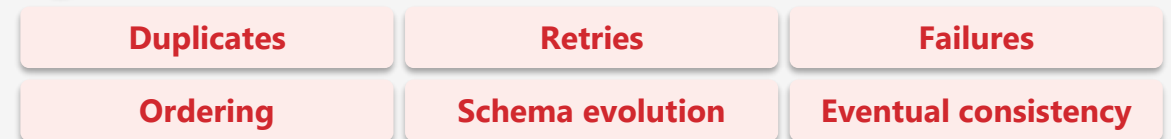
1 Messaging fundamentals



2 Event-driven architecture



3 Production reliability



KEY TAKEAWAY Sending a message is easy — building a reliable distributed business workflow is the real skill.

FROM MODULE 3 TO MODULE 4

One well-built service is ready — this module makes services work together.

MODULE 3 GAVE US

- ✓ A Spring Boot service with a thin REST API
- ✓ Business rules in the service layer
- ✓ Configuration outside the JAR
- ✓ Consistent errors and useful logs
- ✓ One stateless, runnable JAR

NOW Connect Order to Payment, Inventory and Shipping — and build Lab 2.



OUR PATH THROUGH MODULE 4

- 1 Why services must integrate
- 2 Choose synchronous or asynchronous
- 3 Brokers, queues and topics
- 4 Design events, producers and consumers
- 5 Coordinate workflows across services
- 6 Survive duplicates, retries and reordering
- 7 Contracts, the outbox and choosing a pattern

Part 1

Part 1

Part 1

Part 2

Part 2

Part 3

Part 3

KEY TAKEAWAY Module 3 built one service; Module 4 makes independent services work together reliably.

PART 1: MESSAGING FUNDAMENTALS

How services talk without waiting on each other.

Part 1
Messaging fundamentals

Part 2
Event-driven architecture

Part 3
Production reliability

SO FAR

- ✓ One Spring Boot service with a clean REST API
- ✓ Configuration lives outside the JAR
- ✓ Consistent errors and useful logs

NEXT QUESTION

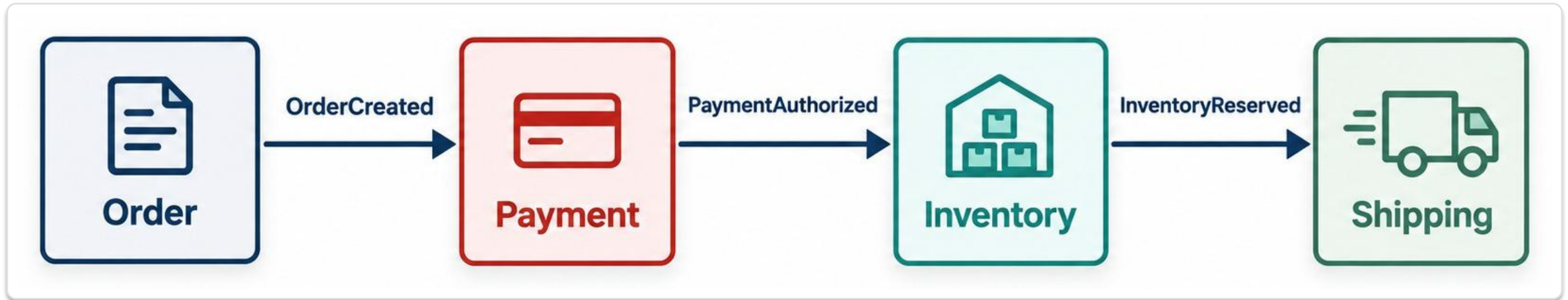
Each service works on its own — how do they work together without becoming tangled?

IN THIS PART

- 1 Why services must integrate
- 2 Synchronous vs. asynchronous
- 3 Messaging and its moving parts
- 4 What a broker adds
- 5 Exercise: when a service is down
- 6 Queues vs. publish/subscribe

WHY MICROSERVICES NEED INTEGRATION

Independent services still exchange information and coordinate workflows.



ONE ORDER MAY NEED

- ✓ Payment authorization
- ✓ Inventory reservation
- ✓ Shipping
- ✓ Customer notification

WHAT MAKES IT HARD

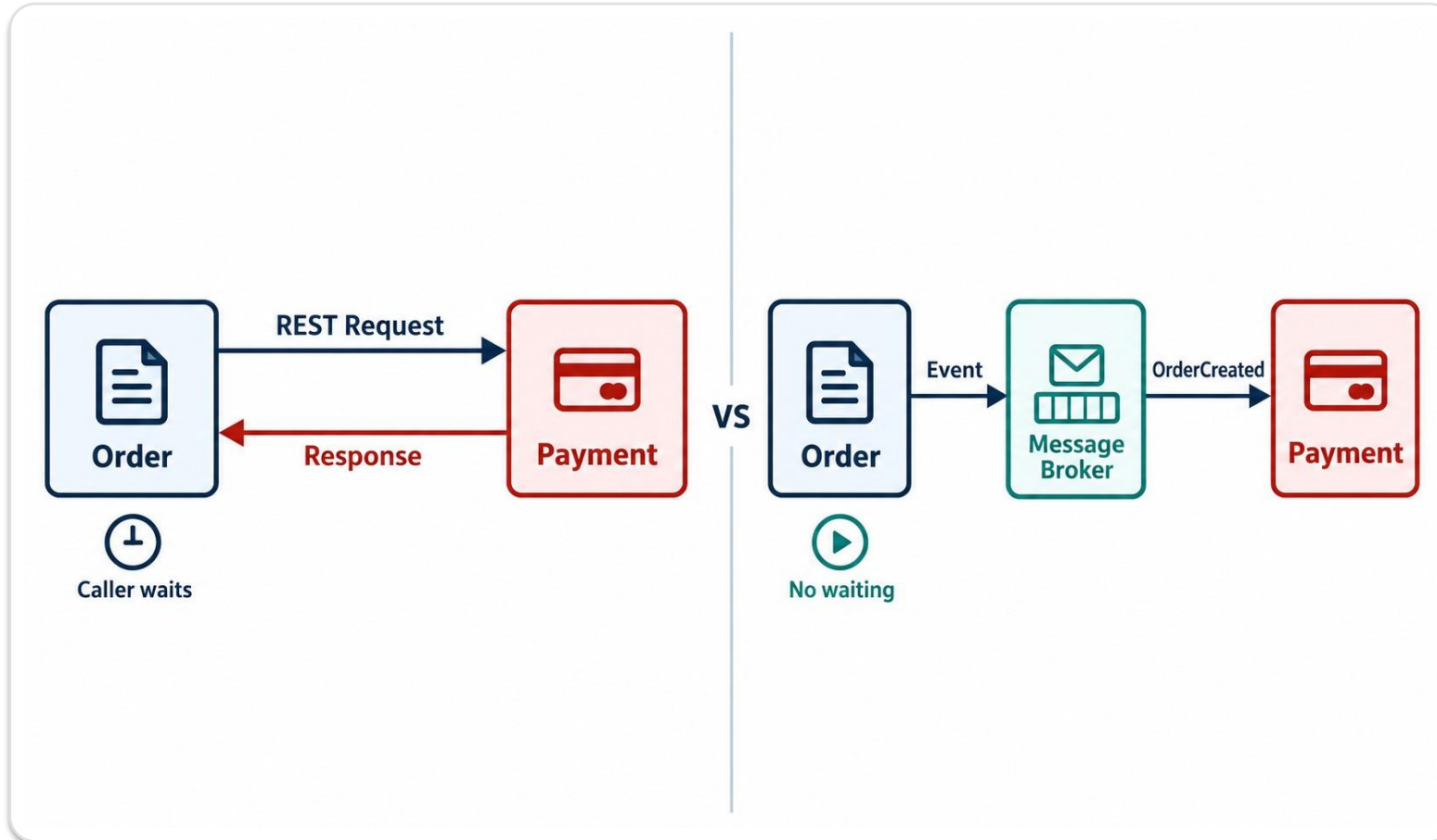
- ! Latency and timeouts
- ! A service that's down
- ! Duplicate messages
- ! Changing contracts

DON'T COLLAPSE IT Moving it all back into Order Service erases the boundaries.

KEY TAKEAWAY Real business processes span services — so services must integrate through well-defined mechanisms.

SYNCHRONOUS VS. ASYNCHRONOUS COMMUNICATION

Request and response compared with message- and event-based communication.



Use synchronous when

The caller needs an answer now — for example, checking stock before accepting an order.

Use asynchronous when

Work can finish later or many services react — for example, the confirmation email.

IF YOU CALL SYNCHRONOUSLY

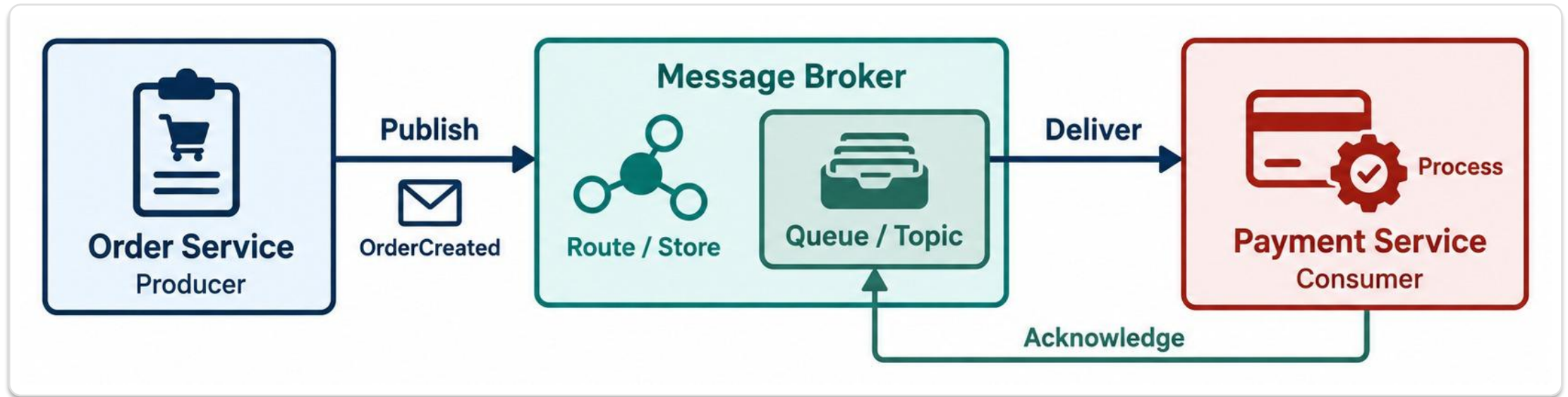
- ✓ Set a timeout on every call
- ✓ Avoid long call chains

DECIDE BY the business interaction, not the technology.

KEY TAKEAWAY Synchronous means request, wait, response; asynchronous means produce and continue.

MESSAGING: THE PARTS AND THE LIFECYCLE

Producer, message, broker, destination and consumer — from publish to acknowledge.



Producer

Creates and sends the message — Order Service.

Message

The information — OrderCreated for order 1001.

Broker and destination

Kafka, RabbitMQ or a cloud service; a queue or a topic.

Consumer

Receives and processes it — Payment Service.

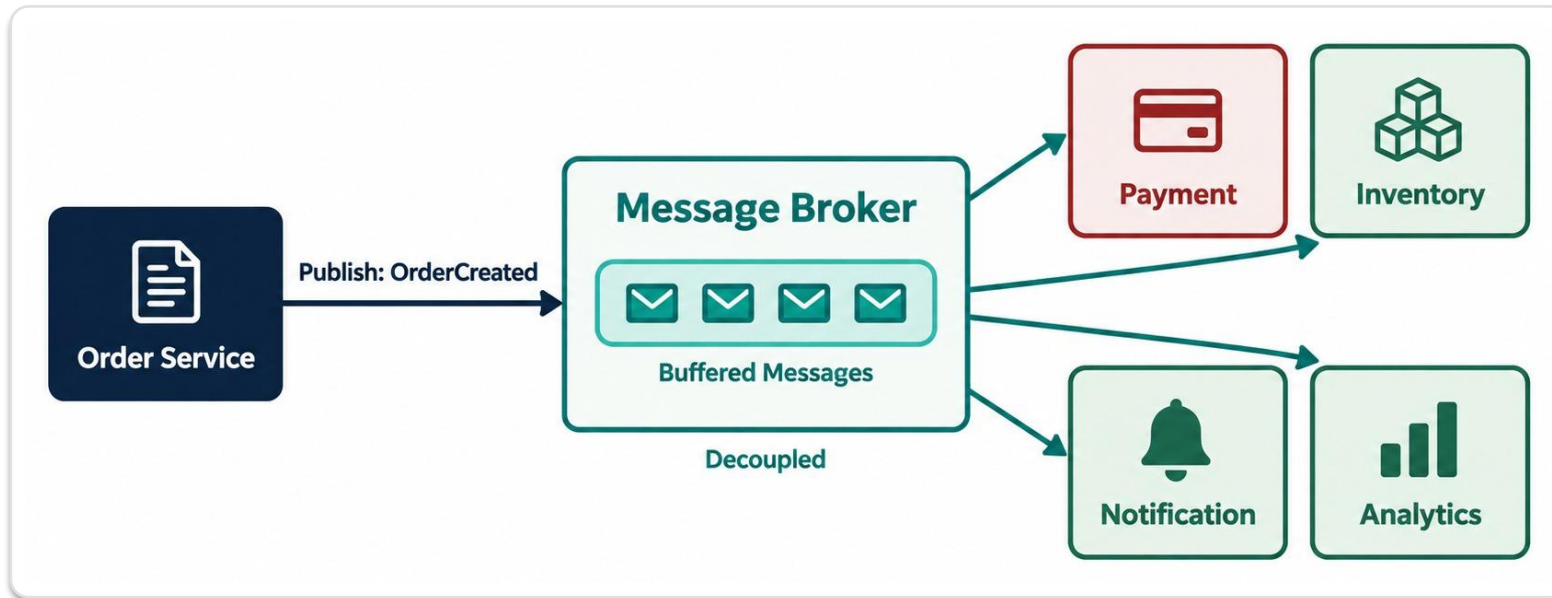
Acknowledge

Records success. No acknowledgment means redelivery.

KEY TAKEAWAY A broker sits between producers and consumers, so they don't need point-to-point knowledge of each other.

MESSAGE BROKERS IN MICROSERVICES

Routing, buffering, decoupling and delivery.



WHAT A BROKER PROVIDES

Routing

Buffering

Durability

Redelivery

Load distribution

Decoupling

Common brokers

Apache Kafka, RabbitMQ, ActiveMQ, Amazon SQS and SNS, Azure Service Bus, Google Pub/Sub.

BUFFERING Inventory down? Messages wait. An HTTP caller would fail now.

KEY TAKEAWAY A broker lets Order publish once, while consumers receive and process independently.

EXERCISE: WHEN NOTIFICATION IS DOWN

Compare a REST call and an event when one service fails.

PRACTICE EXERCISE

10 MIN

SCENARIO

Notification Service is down for 10 minutes during a sale.

Version A: after saving, Order calls POST /notifications over REST.

Version B: after saving, Order publishes OrderCreated; Notification consumes it.

YOUR TASKS

- 1 Trace one order through A and B while Notification is down
- 2 Say what the customer sees in each version
- 3 Say what happens when Notification comes back

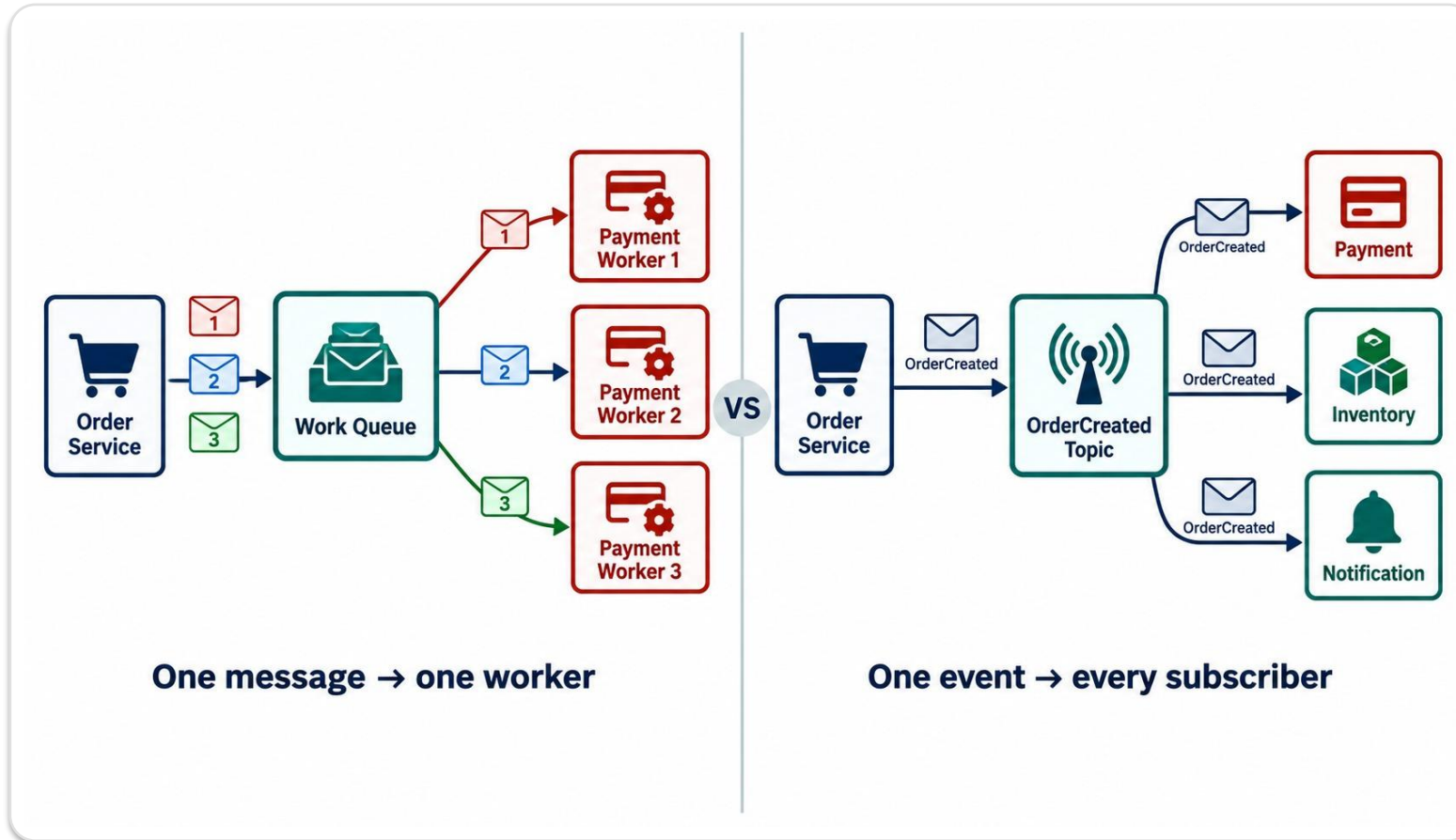
EXPECTED OUTCOME

- ✓ A: the call times out — fail the order or skip the email
- ✓ A: an error, or a missing email
- ✓ B: the order succeeds; the event waits in the broker
- ✓ B: Notification catches up when it recovers
- ✓ B still needs idempotency and lag monitoring

KEY TAKEAWAY A broker turns a failing dependency into delayed work instead of a failed order.

QUEUE VS. PUBLISH/SUBSCRIBE

Competing consumers versus multiple independent subscribers.



Queue — work distribution

GenerateInvoice jobs shared across workers.
Add workers to go faster.

Topic — event distribution

OrderCreated to every subscriber. Add
subscribers without touching Order.

Kafka consumer groups

Instances in one group share the work; each
group gets every event.

TECHNOLOGIES DIFFER Kafka: topics and
groups. RabbitMQ: exchanges and queues.

KEY TAKEAWAY Queues distribute work to one consumer; pub/sub distributes events to many — and technologies differ.

PART 2: EVENT-DRIVEN ARCHITECTURE

From sending messages to designing around business events.

✓ Part 1
Messaging fundamentals

Part 2
Event-driven architecture

Part 3
Production reliability

SO FAR

- ✓ Sync waits; async produces and continues
- ✓ A broker routes, buffers and redelivers
- ✓ Queues share work; topics share events

NEXT QUESTION

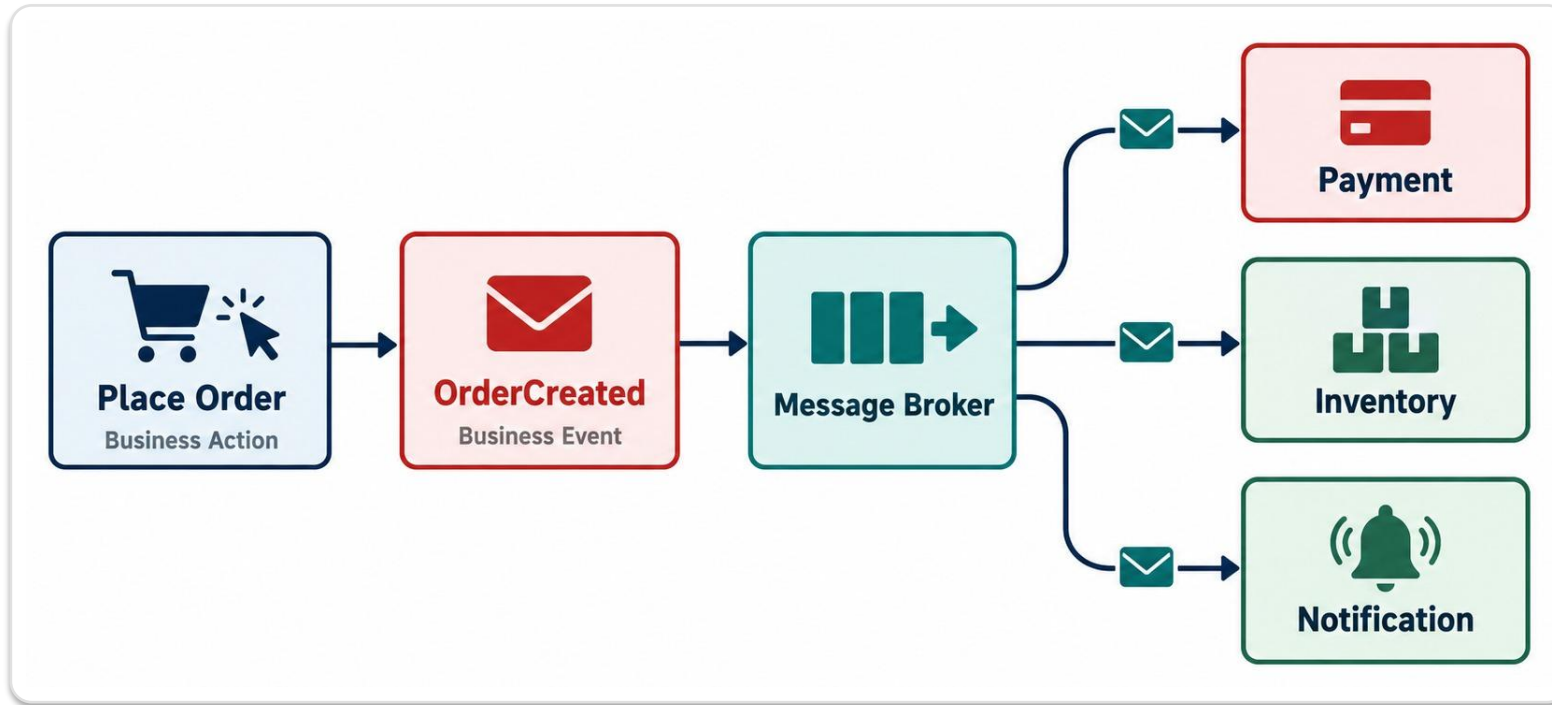
How do we design a system around business events — and who controls the workflow?

IN THIS PART

- 1 Business events and event-driven architecture
- 2 Commands, events and messages
- 3 An event-driven order, step by step
- 4 Producers and consumers — with Kafka code
- 5 Choreography, and when to orchestrate
- 6 Exercise: command or event?

INTRODUCTION TO EVENT-DRIVEN ARCHITECTURE

Services react to business events instead of relying only on direct calls.



FACTS, IN THE PAST TENSE

OrderCreated

PaymentAuthorized

PaymentFailed

InventoryReserved

OrderCancelled

ShipmentCreated

AVOID

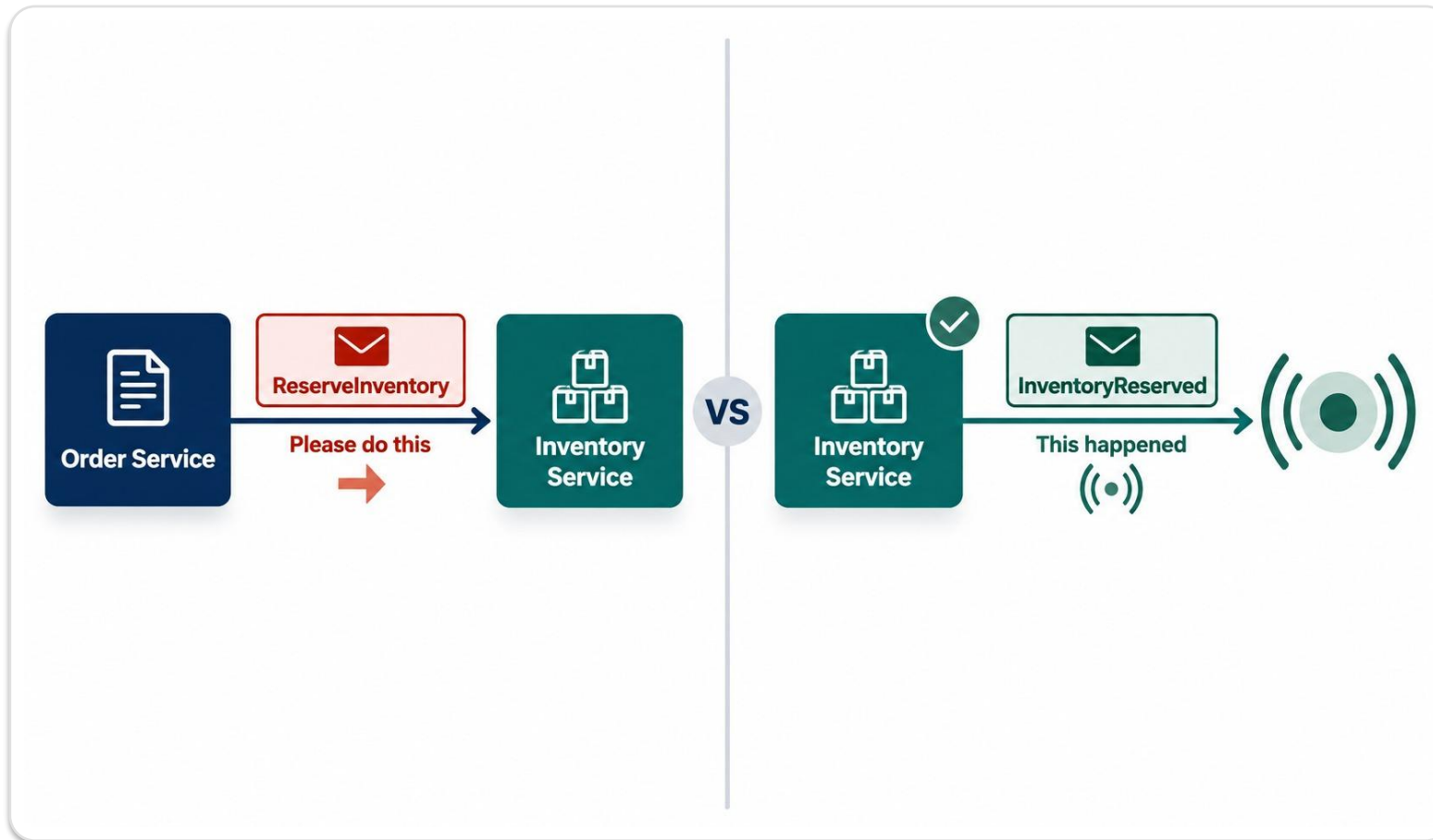
- ❌ DataChanged — too vague
- ❌ PostPayment — sounds like a command

CONSUMERS DECIDE how — and whether — to react.

KEY TAKEAWAY An event is a fact that already happened — consumers choose how to react to it.

COMMANDS, EVENTS, AND MESSAGES

Intent versus facts — and the envelope that carries them.



Command

ProcessPayment, ReserveInventory, ShipOrder.
Intent; usually one handler; it can be refused.

Event

PaymentProcessed, InventoryReserved,
OrderShipped. A fact; many may care; never
changed.

Notification

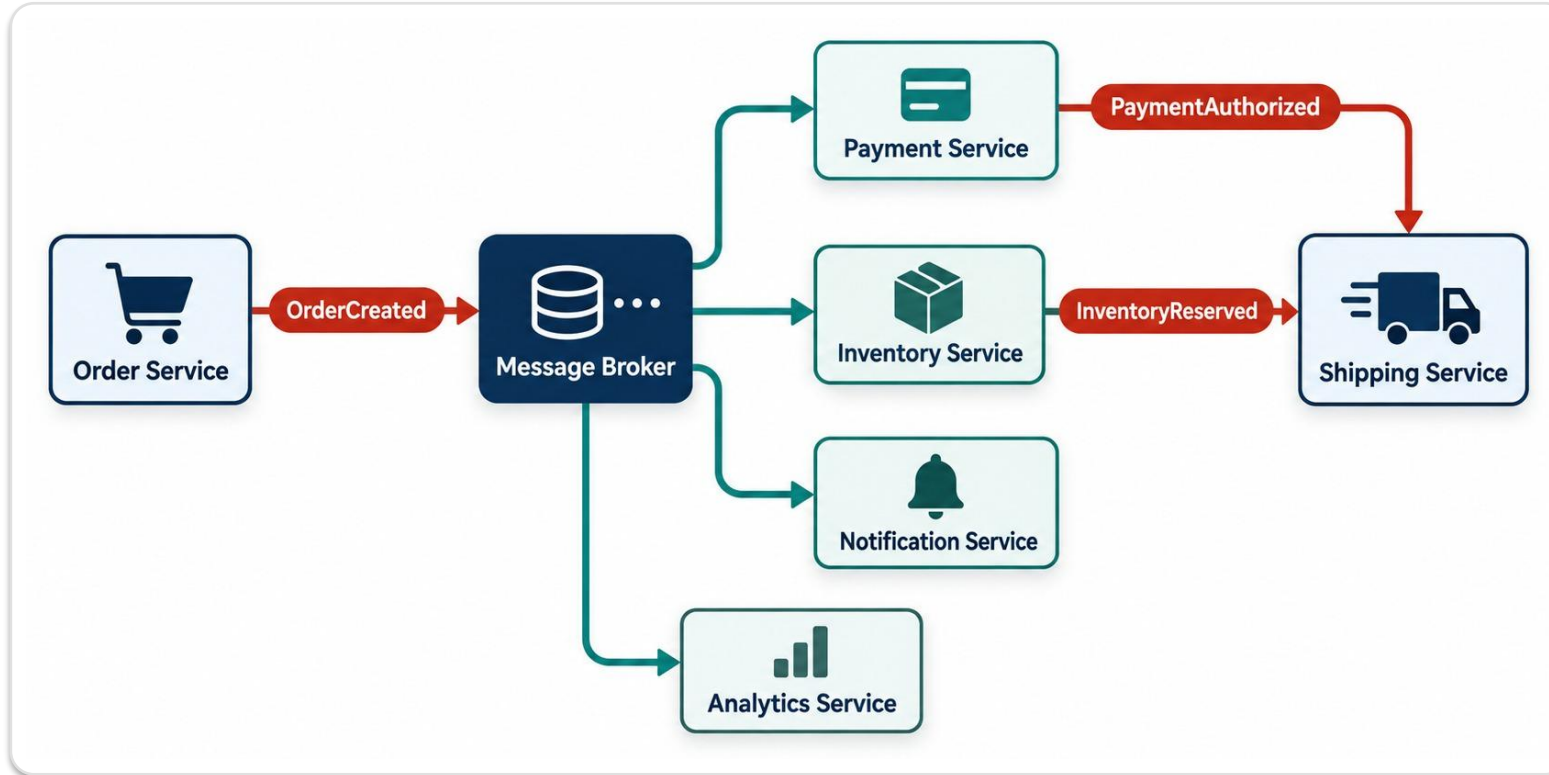
Informs or alerts; no action is required.

MESSAGE the envelope that carries any of
them.

KEY TAKEAWAY Commands ask for action, events report facts, and messages carry either one.

EVENT-DRIVEN ORDER PROCESSING EXAMPLE

OrderCreated → *Payment, Inventory and Notification* → *Shipping*.



WALK THROUGH ONE ORDER

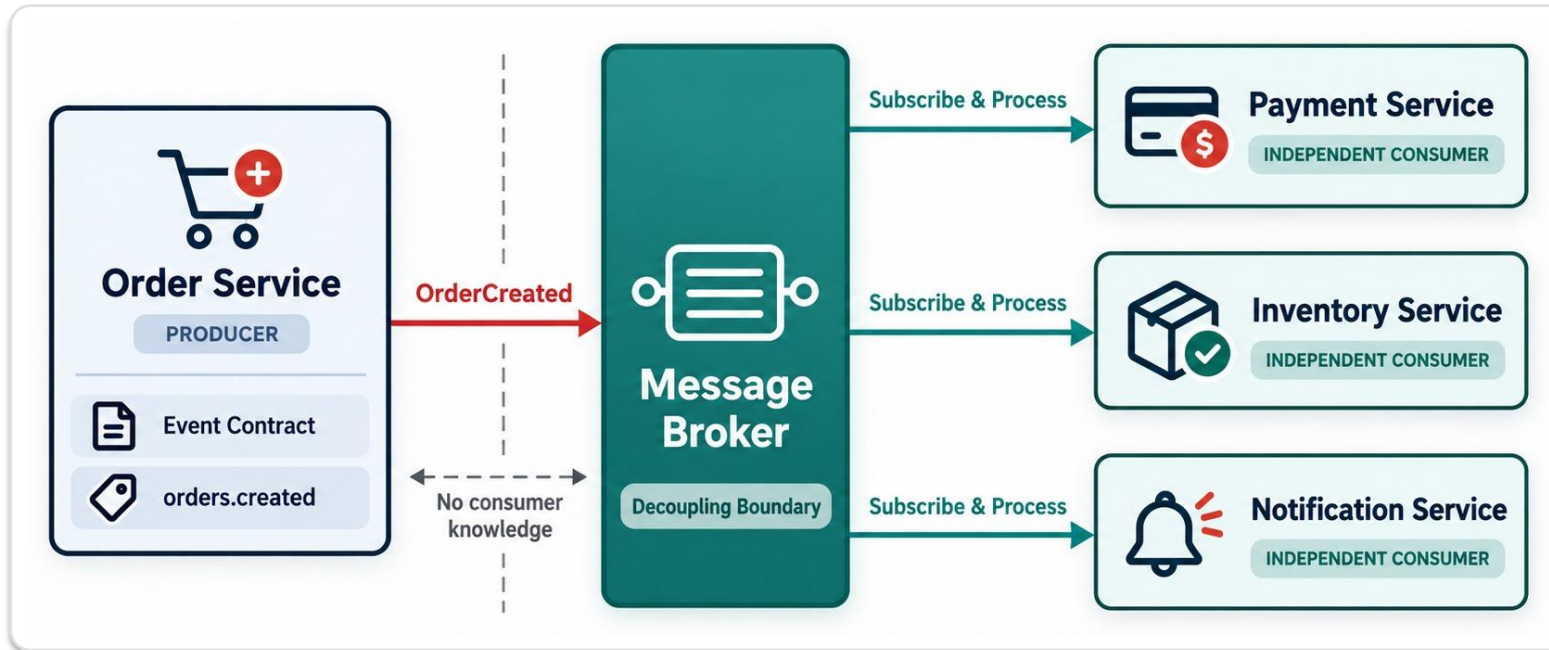
- ✓ POST /orders creates the order
- ✓ Order publishes OrderCreated
- ✓ Payment, Inventory and Notification react
- ✓ They publish PaymentAuthorized and InventoryReserved
- ✓ Shipping starts when both have arrived

CAUTION Publishing events doesn't make the workflow correct by itself.

KEY TAKEAWAY Business rules define the workflow — events carry it from one service to the next.

EVENT PRODUCER AND CONSUMER DESIGN

Clear responsibilities — and no unnecessary knowledge of each other.



✗ Avoid in Order Service

```
if (inventoryServiceExists) ...  
if (analyticsServiceExists) ...
```

Extensibility

Add Analytics as a new subscriber — no change to Order Service.

NOT ZERO COUPLING Everyone still depends on the event contract.

KEY TAKEAWAY Producers publish facts and move on; consumers decide for themselves whether to react.

PRODUCING AND CONSUMING WITH SPRING FOR APACHE KAFKA

KafkaTemplate publishes; @KafkaListener consumes.

Order Service — producer

```
@Service
public class OrderService {
    private final KafkaTemplate<String, OrderCreated> kafka;

    public OrderResponse place(CreateOrderRequest req) {
        Order order = orders.save(Order.from(req));
        kafka.send("orders.created",
            order.getId().toString(), OrderCreated.from(order));
        return OrderResponse.from(order);
    }
}
```

Key = orderId

Events for one order land in one partition, in order.

groupId

Payment instances share the work; other groups get every event.

✓ Payment Service — consumer

```
@Component
public class OrderCreatedListener {

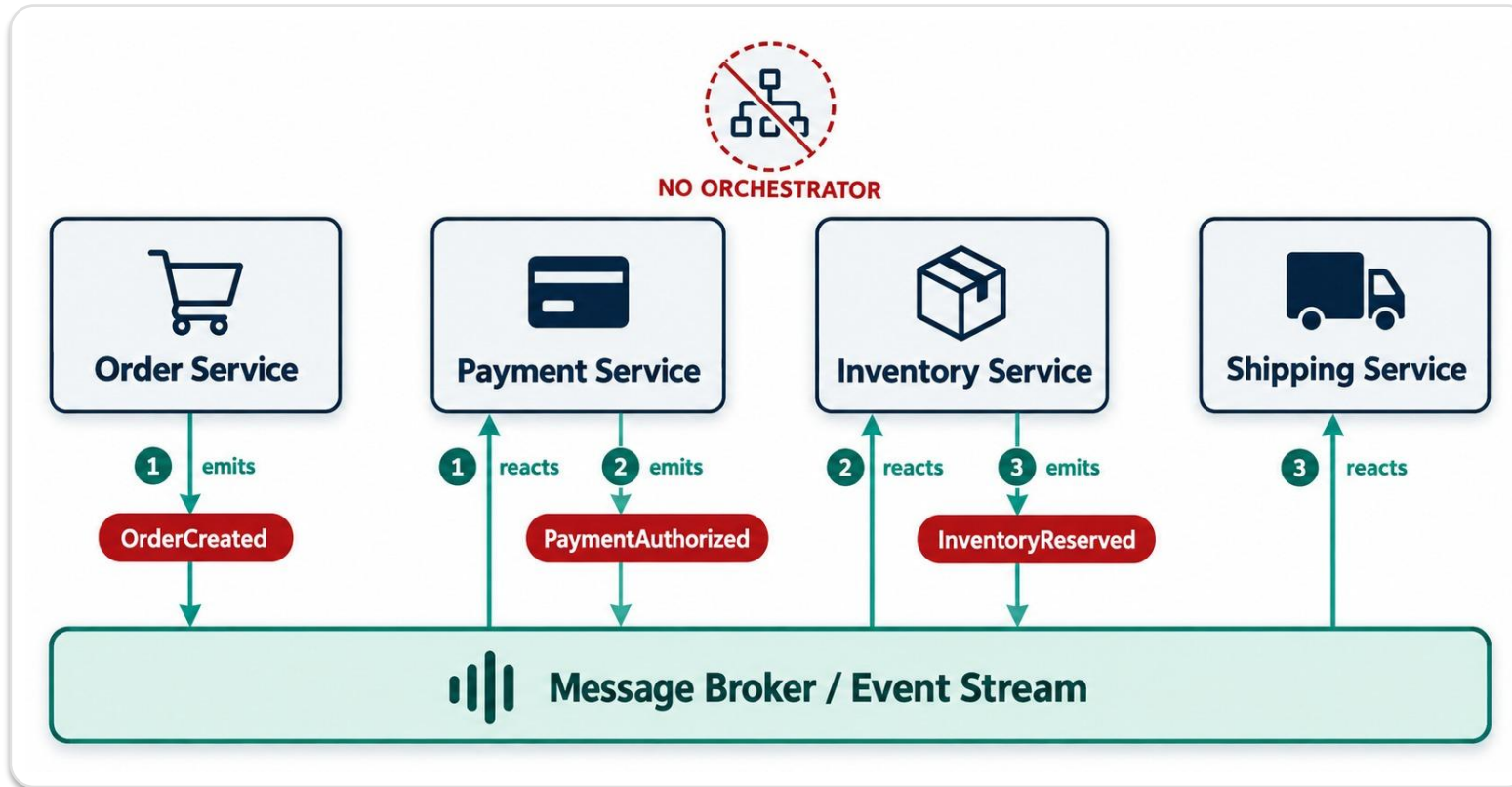
    @KafkaListener(topics = "orders.created",
        groupId = "payment-service")
    public void on(OrderCreated event) {
        payments.authorize(event.orderId(), event.total());
    }
}
```

GAP Saved but not sent? The outbox in Part 3 fixes it.

KEY TAKEAWAY Publish the event after the business change, keyed by the entity — consumers subscribe by topic and group.

EVENT CHOREOGRAPHY

Services react to events without a central workflow controller.



BENEFITS

- ✓ Decentralized
- ✓ Services stay autonomous
- ✓ No central workflow dependency

CHALLENGES

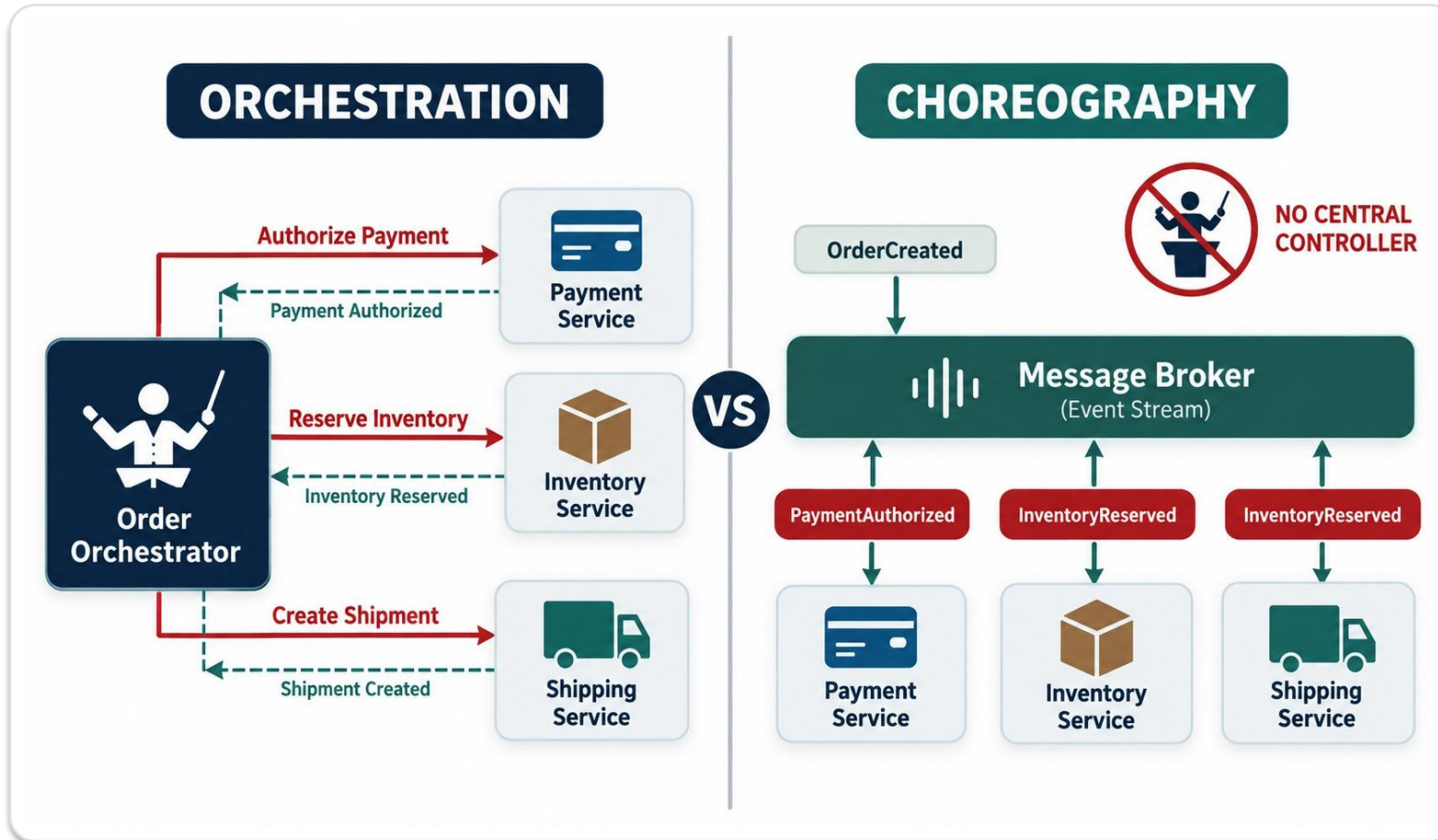
- ✗ The whole flow is hard to see
- ✗ Troubleshooting takes longer
- ✗ Long event chains get fragile

LIKE A DANCE each service responds to the others' signals.

KEY TAKEAWAY In choreography, each service reacts and publishes the next event — nobody controls the whole flow.

ORCHESTRATION OR CHOREOGRAPHY: HOW TO CHOOSE

A central coordinator versus a decentralized event-driven workflow.



Choose orchestration when

Many steps, branching decisions, compensation, and a need to see the whole flow in one place.

Choose choreography when

Few steps, independent reactions, and teams that must evolve separately.

MIX THEM Orchestrate the core order saga; choreograph side reactions like email and analytics.

KEY TAKEAWAY Neither is universally better — choose by workflow complexity and organizational needs.

EXERCISE: COMMAND OR EVENT?

Sort message names from a design review — and fix the weak ones.

PRACTICE EXERCISE

10 MIN

SCENARIO

The team proposed these message names:

ChargeCard
CardCharged
SendWelcomeEmail
AccountOpened
ReserveStock
StockReserved
DataChanged
OrderShipped

YOUR TASKS

- 1 Label each one a command or an event
- 2 For two events, name the services that might react
- 3 Rename anything vague — and say what's wrong with it

EXPECTED OUTCOME

- ✓ Commands: ChargeCard, SendWelcomeEmail, ReserveStock
- ✓ Events: CardCharged, AccountOpened, StockReserved, OrderShipped
- ✓ AccountOpened → Notification, Analytics
- ✓ DataChanged is vague → CustomerAddressChanged

KEY TAKEAWAY Commands are imperative requests to one handler; events are specific past-tense facts for anyone.

PART 3: PRODUCTION RELIABILITY

Sending a message is easy; processing it correctly under failure is the real work.

✓ Part 1
Messaging fundamentals

✓ Part 2
Event-driven architecture

Part 3
Production reliability

SO FAR

- ✓ Events are past-tense business facts
- ✓ Producers publish; consumers subscribe independently
- ✓ Choreography or orchestration, chosen per workflow

NEXT QUESTION

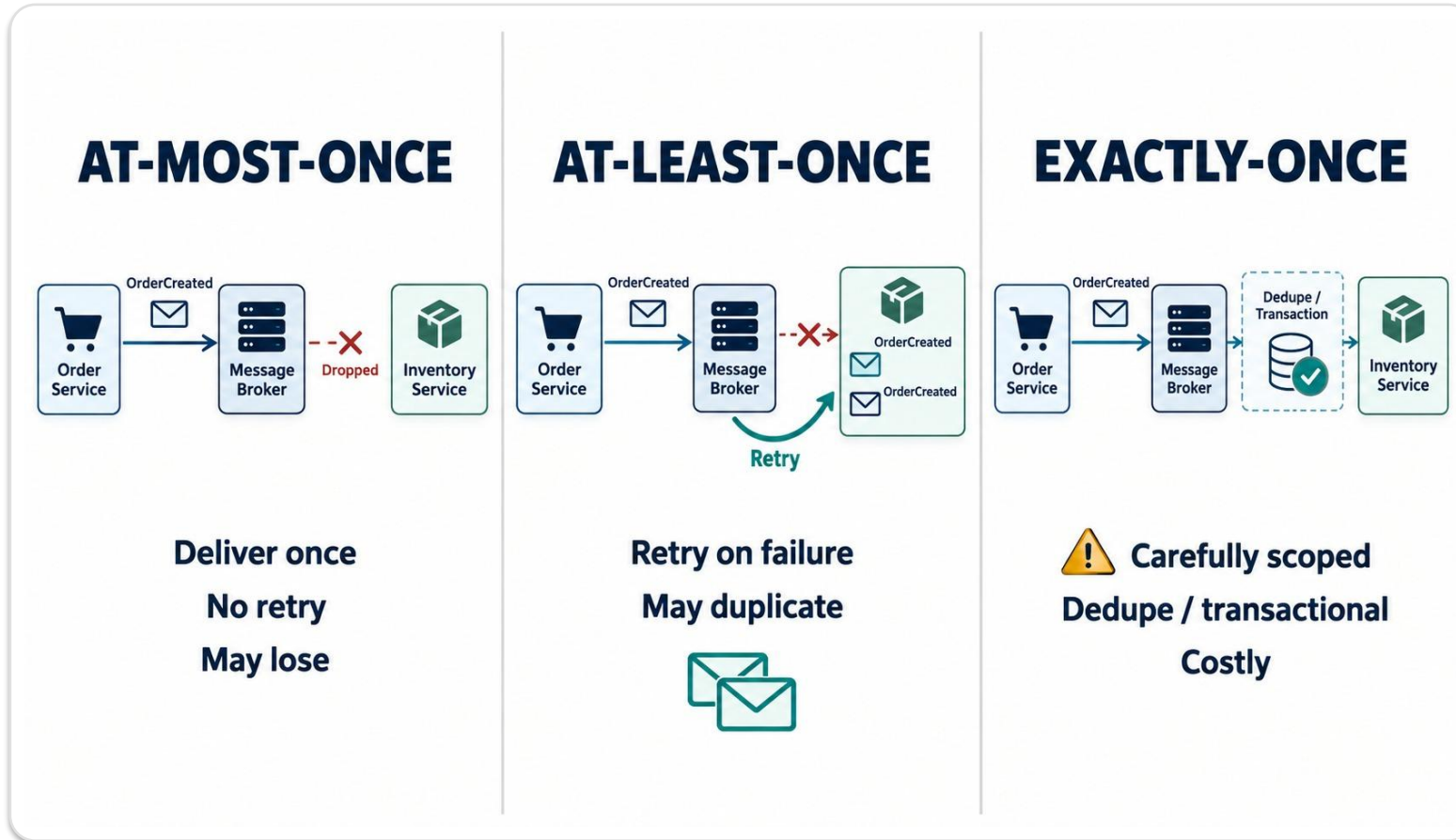
Messages get lost, duplicated, delayed and reordered — how do we stay correct?

IN THIS PART

- 1 Delivery guarantees
- 2 Idempotent consumers, retries and dead letters
- 3 Exercise 2.3: idempotency and DLT
- 4 Eventual consistency and the outbox
- 5 Event contracts and Exercise 2.2
- 6 Kafka vs. RabbitMQ, choosing, the full picture

MESSAGE DELIVERY SEMANTICS

At-most-once, at-least-once — and what “exactly-once” really claims.



At-most-once

No duplicates, but messages may be lost. Fine for metrics — never for payments.

At-least-once

Nothing lost, but duplicates happen. The usual choice for orders and payments.

Exactly-once

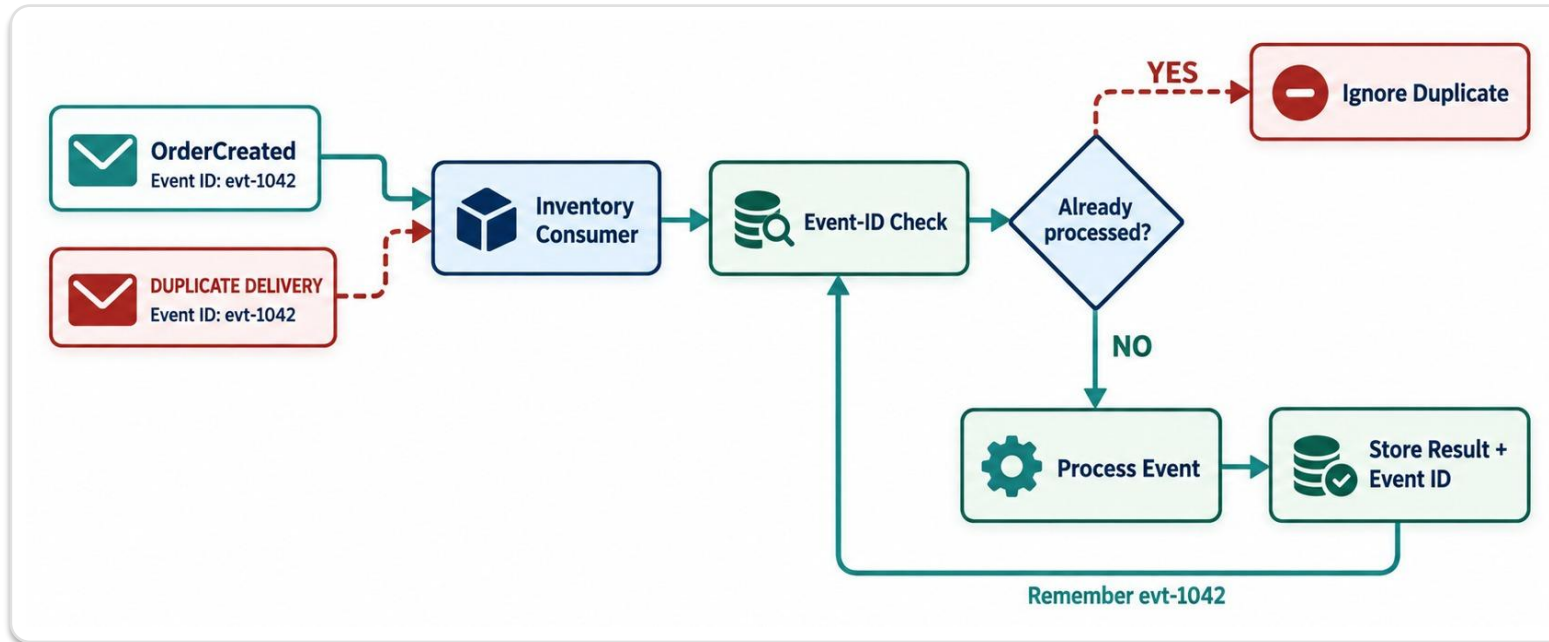
Scoped to the broker, such as Kafka transactions — not to emails or external APIs.

ASK Exactly once where — and under what failure?

KEY TAKEAWAY At-least-once delivery is common — so consumers must be ready for duplicates.

IDEMPOTENT MESSAGE CONSUMERS

Handle duplicate delivery without duplicating the business effect.



✗ **Not idempotent**

```
balance = balance.add(event.amount());
```

TECHNIQUES

Processed event IDs

Unique business keys

Upserts

State checks

DB constraints

RULE A redelivered payment must never charge twice.

KEY TAKEAWAY A redelivered payment message must never charge the customer twice.

AN IDEMPOTENT CONSUMER IN CODE

Remember every processed event ID in the same transaction as the business change.

✓ PaymentListener.java

```
@KafkaListener(topics = "orders.created",
                groupId = "payment-service")
@Transactional
public void on(OrderCreated event) {
    if (processed.existsById(event.eventId())) {
        log.info("Duplicate ignored eventId={}", event.eventId());
        return;
    }
    payments.record(event.orderId(), event.total());
    processed.save(new ProcessedEvent(event.eventId()));
}
```

processed_events table

```
CREATE TABLE processed_events (
    event_id      VARCHAR(64) PRIMARY KEY,
    processed_at  TIMESTAMP NOT NULL
                  DEFAULT CURRENT_TIMESTAMP
);

-- The primary key rejects a second insert,
-- even when two consumers race.
```

One transaction

The payment row and the processed ID commit together.

Primary key

The database enforces "once", even under a race.

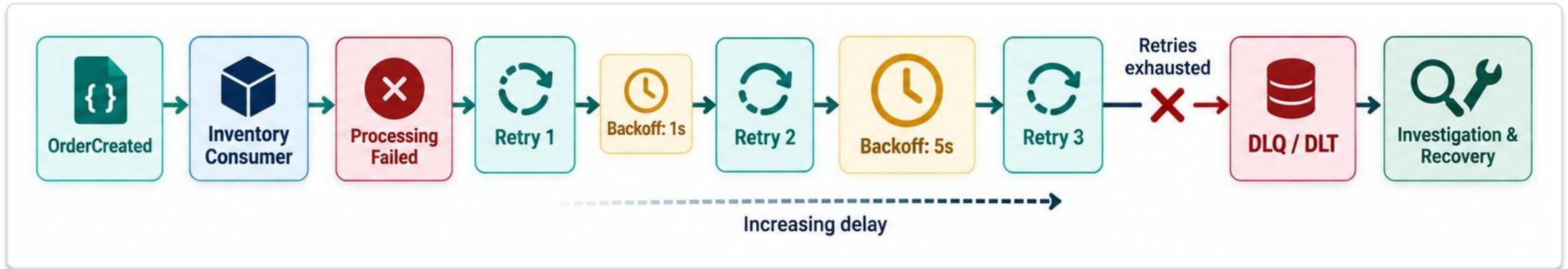
Lab 2

processed_events keyed by event_id — the same pattern.

KEY TAKEAWAY Check, act and record the event ID in one transaction — duplicates then become harmless no-ops.

RETRIES AND DEAD-LETTER HANDLING

Transient failure → retry with backoff → dead letter → investigation and recovery.



Transient failure

The database is briefly down. Retrying with backoff may well succeed.

Persistent failure

Bad structure or a rule that always rejects. Send it to the dead-letter topic — don't loop.

After the dead letter

Alert, inspect, fix, then replay on purpose. Never invent a success.

KEY TAKEAWAY Retry transient failures with backoff; send what still fails to a dead-letter destination.

RETRIES AND DEAD LETTERS IN SPRING KAFKA

One error-handler bean: back off, give up, publish to the .DLT topic.

✓ KafkaErrorConfig.java

```
@Bean
DefaultErrorHandler errors(KafkaTemplate<?, ?> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template);

    var backoff = new ExponentialBackOffWithMaxRetries(3);
    backoff.setInitialInterval(1_000); // 1s, 2s, 4s
    backoff.setMultiplier(2.0);

    var handler = new DefaultErrorHandler(recoverer, backoff);
    handler.addNotRetryableExceptions(
        IllegalArgumentException.class); // bad data
    return handler;
}
```

What lands on the dead-letter topic

Topic: orders.created.DLT

Headers added by Spring Kafka:

```
kafka_dlt-original-topic    orders.created
kafka_dlt-original-offset   4182
kafka_dlt-exception-message
    Invalid total: -20.00
```

Next: alert on DLT depth,
fix the cause, replay on purpose

Backoff

1s, 2s, 4s — a struggling dependency gets time to recover.

Not retryable

Bad data skips the retries and goes straight to the DLT.

Lab 2

Poison messages land on transactions.submitted.DLT.

KEY TAKEAWAY Limit retries with backoff, skip retries for bad data, and park failures on a dead-letter topic.

EXERCISE 2.3 — DESIGN IDEMPOTENCY AND DLQ BEHAVIOUR

Day 2 checkpoint C: duplicate detection and failure routing.

OFFICIAL CHECKPOINT C

10 MIN

curriculum/day-02/exercises/exercise-2.3-idempotency-dlq.md

SCENARIO

Lab 2's Transaction Service consumes TransactionSubmitted from the transactions.submitted topic.

Design what happens to duplicates and poison messages before you code them.

YOUR TASKS

- 1 Same eventId twice: the second consume is a no-op (event_id already in processed_events)
- 2 Poison or unreadable JSON: retry, then dead-letter — never loop forever
- 3 A dead-lettered message must not invent a successful money-movement outcome
- 4 Lab 5 Risk Assessment will consume the same topic — design for that

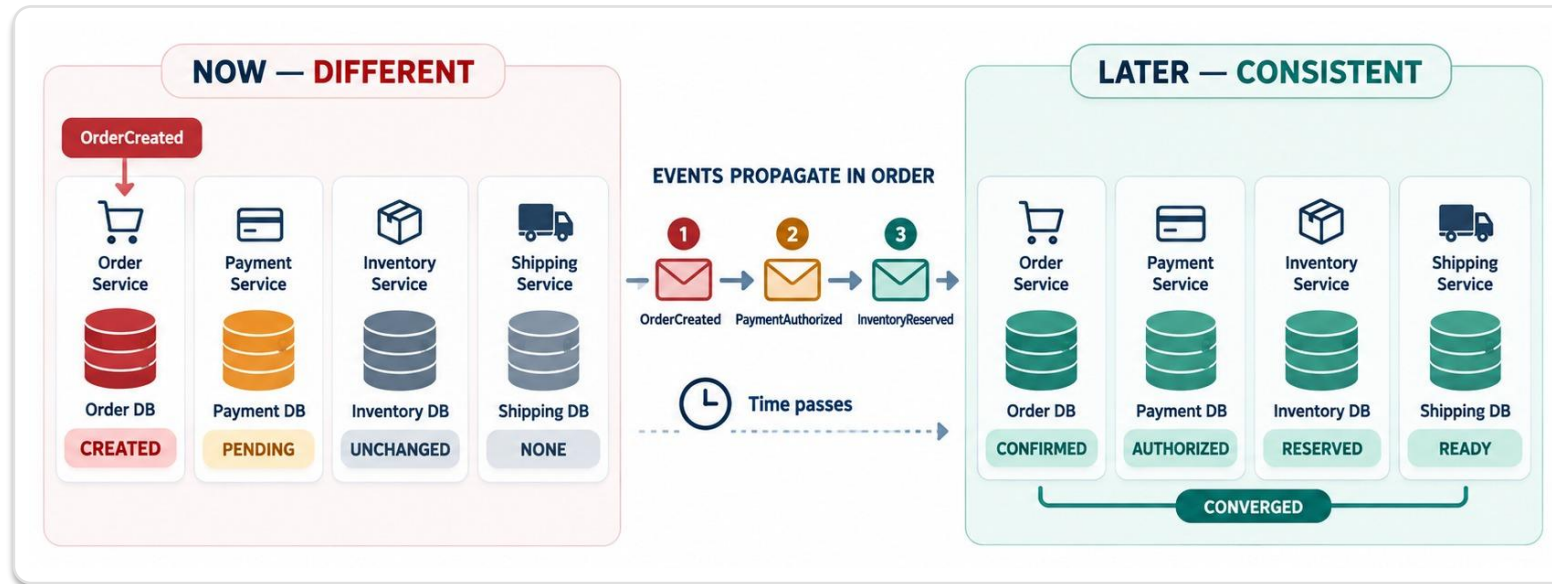
EXPECTED OUTCOME

- ✓ Duplicate eventId → no-op
- ✓ Poison message → transactions.submitted.DLT
- ✓ A DLT message never becomes a success
- ✓ A second consumer group fits without changes

KEY TAKEAWAY You can explain duplicate detection and dead-letter routing before you code them.

ORDERING AND EVENT CONSISTENCY

Event ordering, eventual consistency and stale state.



Out of order

OrderCancelled can arrive before OrderCreated. Consumers must cope.

Kafka ordering

Guaranteed within a partition only — so key events by orderId.

DESIGNED, NOT HOPED FOR Plan for stale views, compensation and reconciliation.

KEY TAKEAWAY Services converge over time — so design for stale views and out-of-order events.

SAVE AND PUBLISH SAFELY: THE TRANSACTIONAL OUTBOX

A database commit and a broker send can't be one atomic step — unless you use an outbox.

✗ Dual write — the event can be lost

```
public OrderResponse place(CreateOrderRequest req) {  
    Order order = orders.save(Order.from(req)); // commits  
  
    // crash or broker outage here:  
    // the order exists, but no OrderCreated event  
    kafka.send("orders.created", OrderCreated.from(order));  
    return OrderResponse.from(order);  
}
```

✓ Outbox — one local transaction

```
@Transactional  
public OrderResponse place(CreateOrderRequest req) {  
    Order order = orders.save(Order.from(req));  
    outbox.save(OutboxEvent.of("orders.created",  
        order.getId(), OrderCreated.from(order)));  
    return OrderResponse.from(order);  
} // both rows commit, or neither does
```

Relay

A scheduler or CDC tool such as Debezium publishes new outbox rows.

At-least-once

The relay may send twice — consumers stay idempotent.

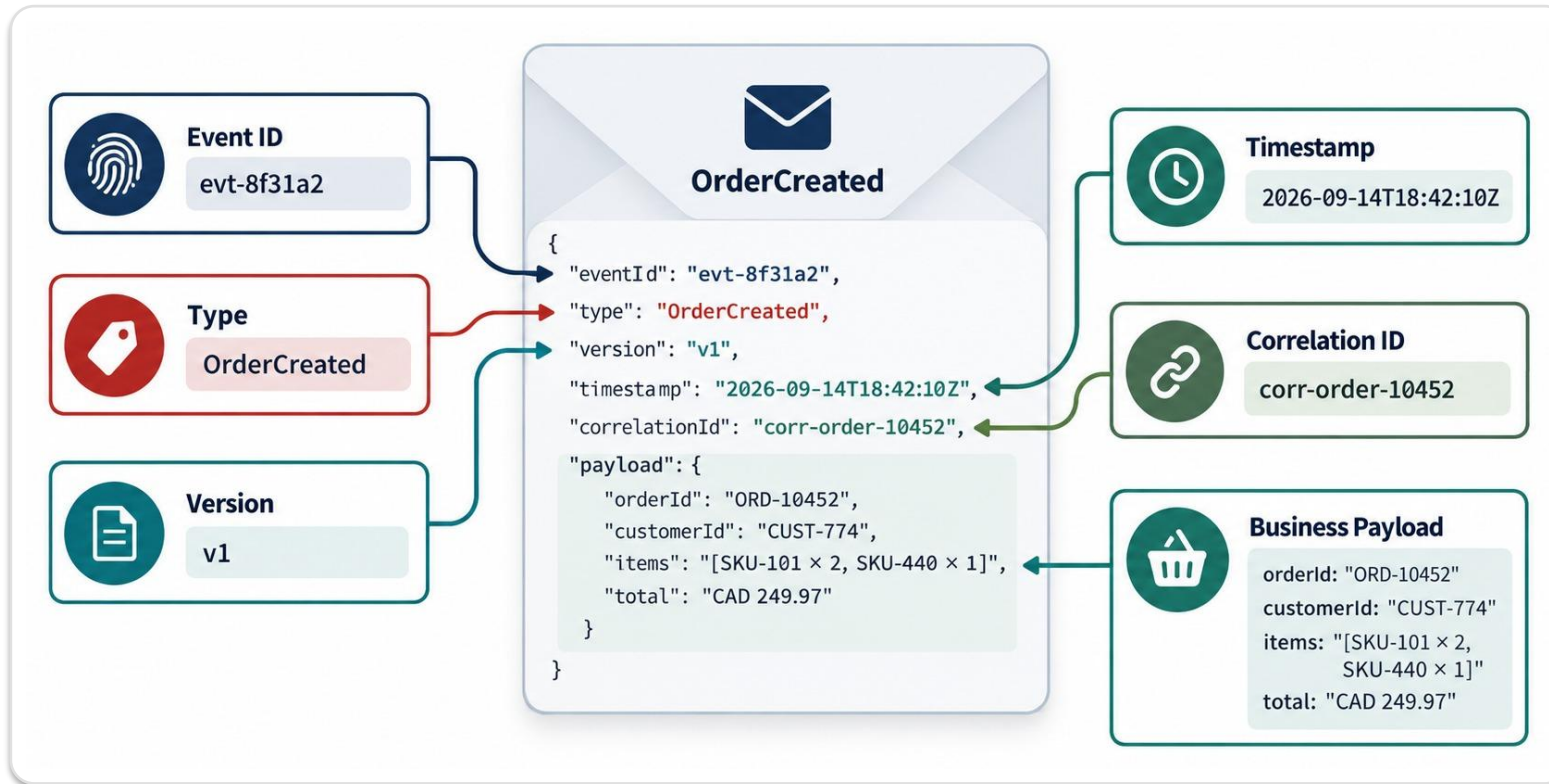
In the labs

Lab 2 saves, then publishes. The outbox is the production hardening step.

KEY TAKEAWAY Write the business change and the event in one local transaction — a relay publishes it afterwards.

EVENT SCHEMA AND CONTRACT DESIGN

Payloads, IDs, timestamps, metadata, versioning and compatibility.



COMPATIBLE CHANGES

- ✓ Add an optional field
- ✓ Add a field with a default

BREAKING CHANGES

- ✗ Remove or rename a field
- ✗ Change a field's type

Thin or full event?

Thin events carry IDs and consumers call back; full events carry the data they need.

NEVER INCLUDE card numbers, SSNs, passwords or tokens.

KEY TAKEAWAY Once other services consume an event, its schema is a distributed contract — evolve it carefully.

EXERCISE 2.2 — SKETCH THE TRANSACTIONSUBMITTED FLOW

Day 2 checkpoint B: producer, consumer and correlation ID.

OFFICIAL CHECKPOINT B

10 MIN

curriculum/day-02/exercises/exercise-2.2-event-flow.md

SCENARIO

Lab 2 builds the banking Transaction Service on port 8082. A client POSTs a transaction, and Account Service confirms the account is ACTIVE over REST.

Scope: Account, Transaction and Risk Assessment only.

YOUR TASKS

- 1 After a valid REST create: persist RECEIVED, then publish TransactionSubmitted
- 2 Envelope: eventType, eventVersion, eventId, correlationId, occurredAt, payload
- 3 Copy X-Correlation-Id from the HTTP request onto the event and into logs
- 4 Lab 2 implements TransactionSubmitted only; Approved, HeldForReview and Declined come later

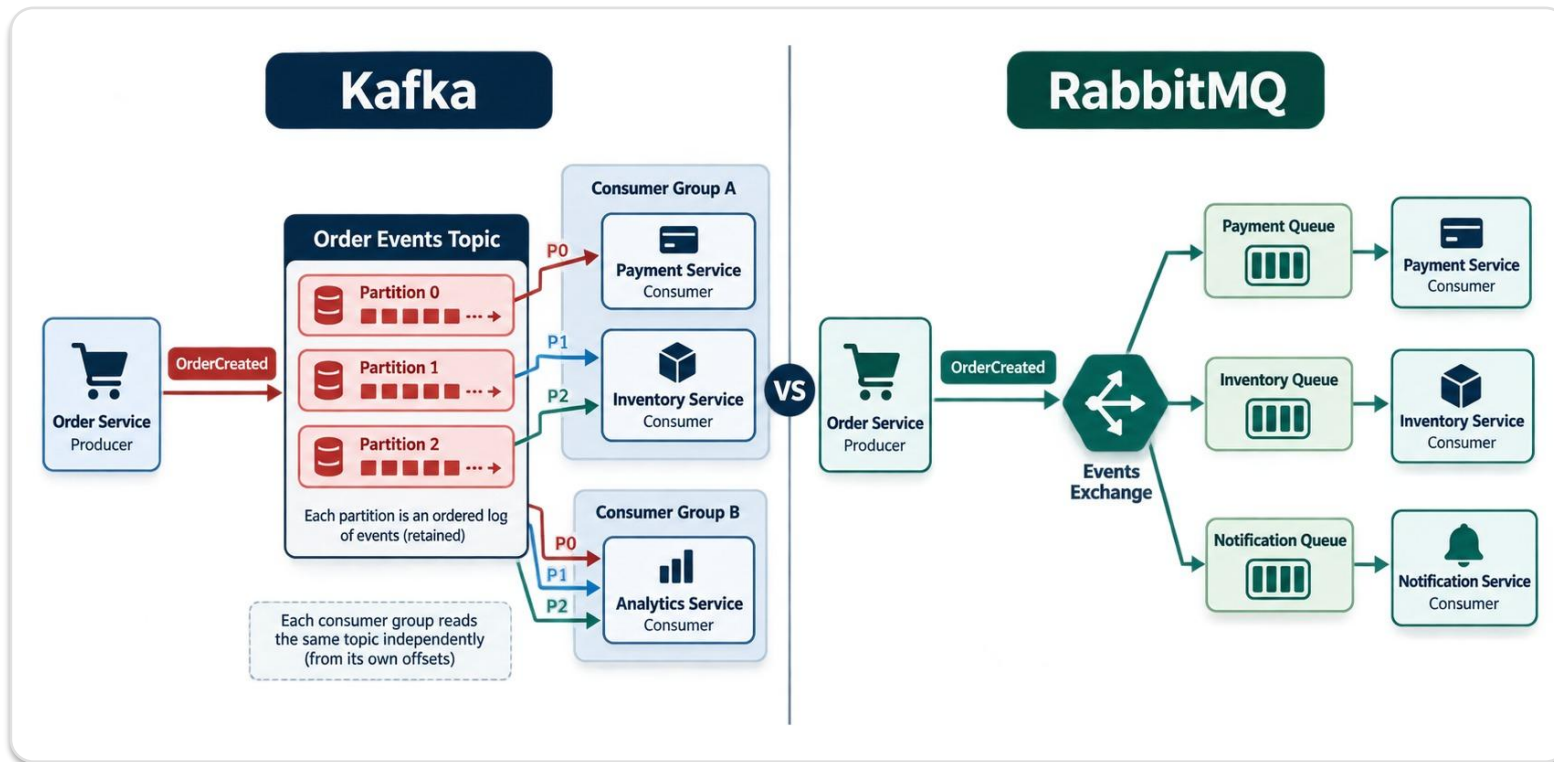
EXPECTED OUTCOME

- ✓ A sequence sketch: REST → DB → Kafka → consumer
- ✓ The correlation ID on every hop
- ✓ All six envelope fields
- ✓ Only TransactionSubmitted in Lab 2

KEY TAKEAWAY A sequence sketch from REST to DB to Kafka to consumer, with the correlation ID on every hop.

KAFKA VS. RABBITMQ — CONCEPTUAL COMPARISON

An event-streaming log platform versus message-broker and queue patterns.



Apache Kafka

A durable, replayable log. Topics with partitions; consumer groups track offsets. Event streams at high throughput.

RabbitMQ

A message broker. Exchanges route to queues; consumers acknowledge. Work queues and flexible routing.

In Spring

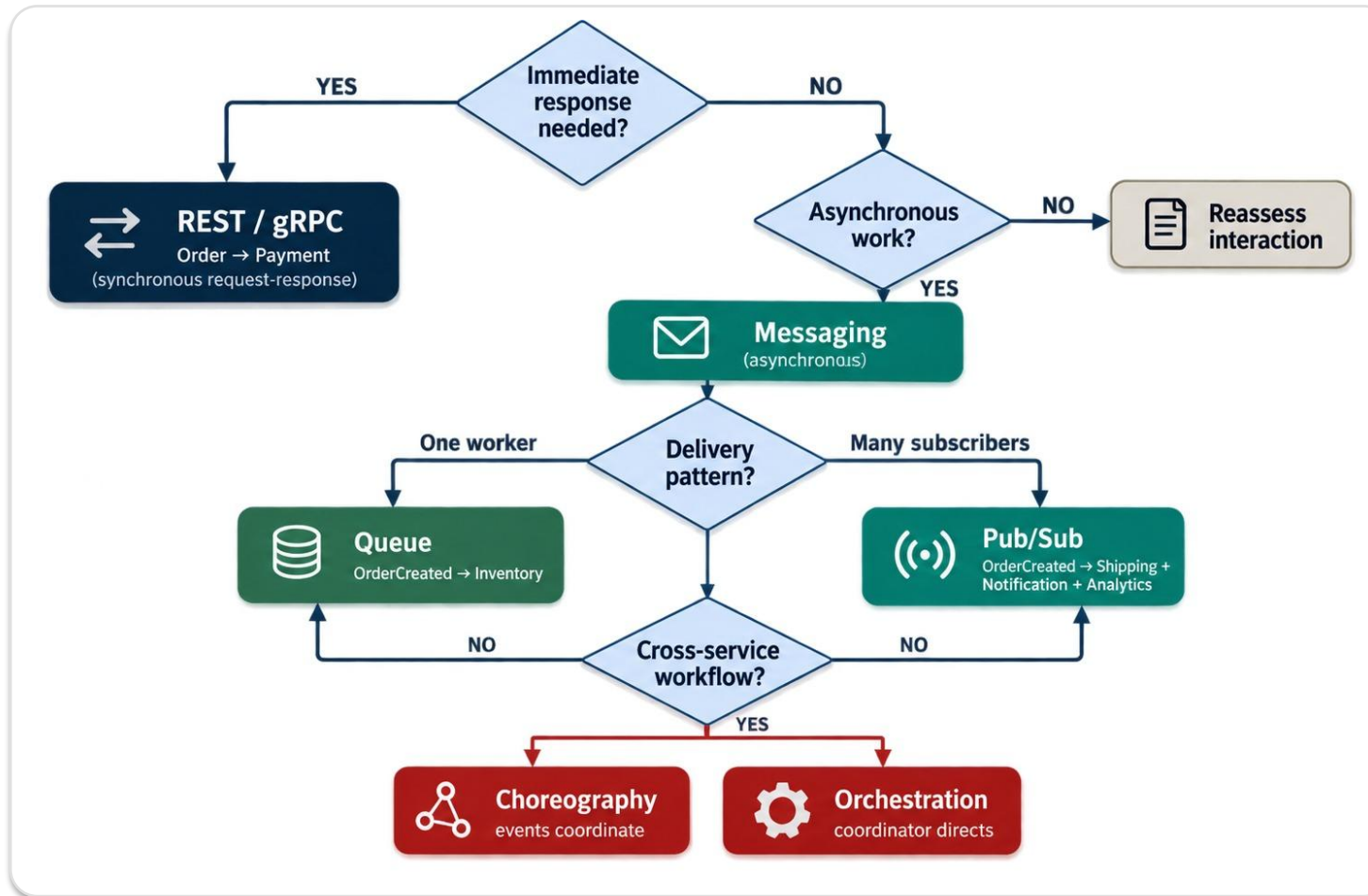
KafkaTemplate with `@KafkaListener`;
RabbitTemplate with `@RabbitListener`.

ASK which model you need — not which is better.

KEY TAKEAWAY Kafka isn't simply a faster RabbitMQ — their architectures and usage models differ.

CHOOSING AN INTEGRATION PATTERN

When to use REST, queues, pub/sub, events, orchestration — or a combination.



Get customer details

Needs an answer now → REST

Generate an invoice

One worker, in the background → queue

OrderCreated

Many consumers → publish/subscribe

Order saga with refunds

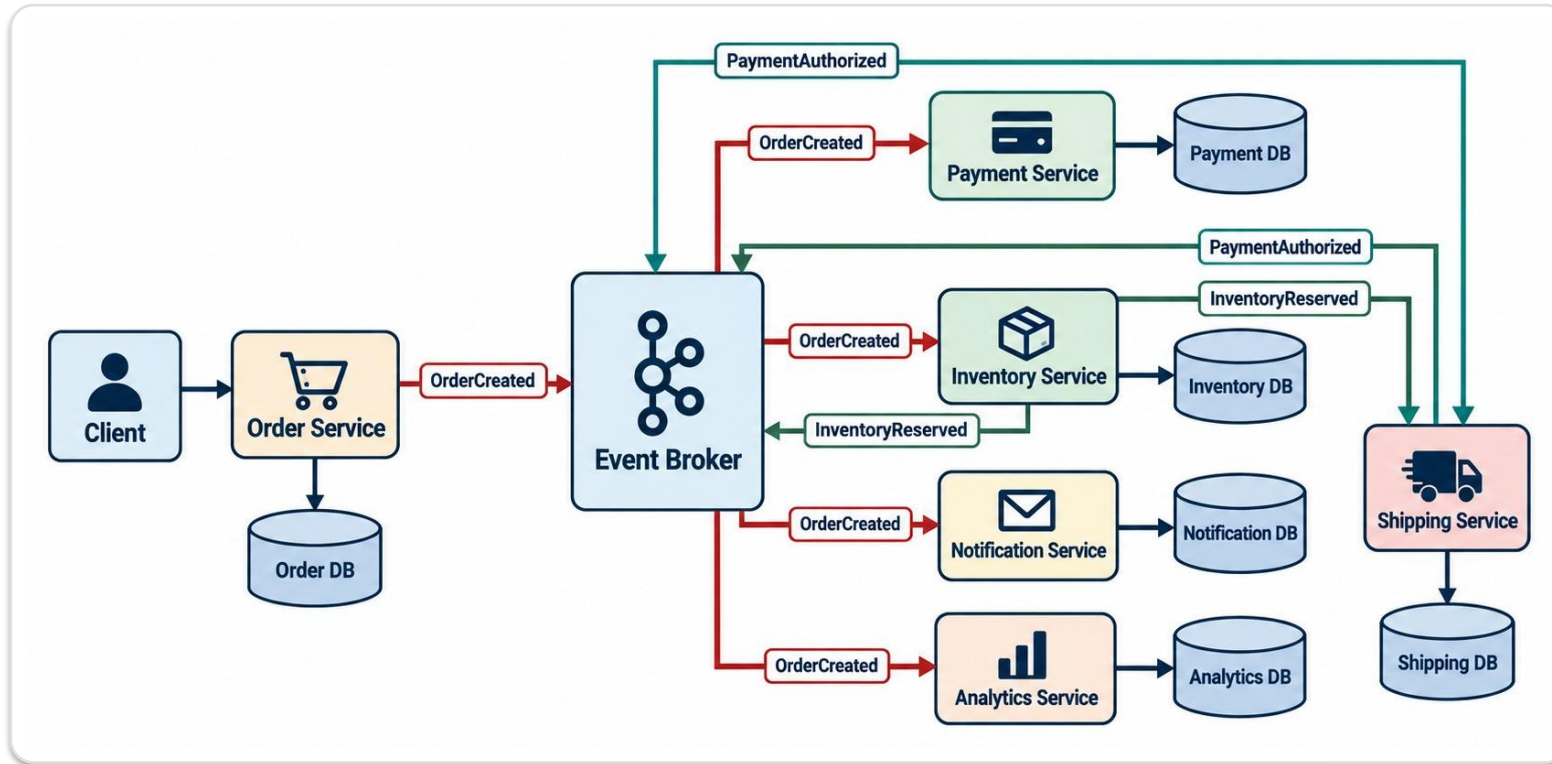
Controlled steps and compensation → orchestration

MOST SYSTEMS combine several patterns.

KEY TAKEAWAY Choose by business semantics, reliability and coupling — not by technology popularity.

END-TO-END EVENT-DRIVEN MICROSERVICES ARCHITECTURE

Order → Broker → Payment, Inventory, Notification → Shipping — each with its own data.



PRODUCTION RELIABILITY

Acknowledgments

Idempotency

Retries + backoff

Dead-letter topics

Schema versioning

Outbox

Correlation IDs

Consumer-lag alerts

Watch in production

Consumer lag, dead-letter depth, retry counts and end-to-end latency.

EVERYTHING HERE came from this module.

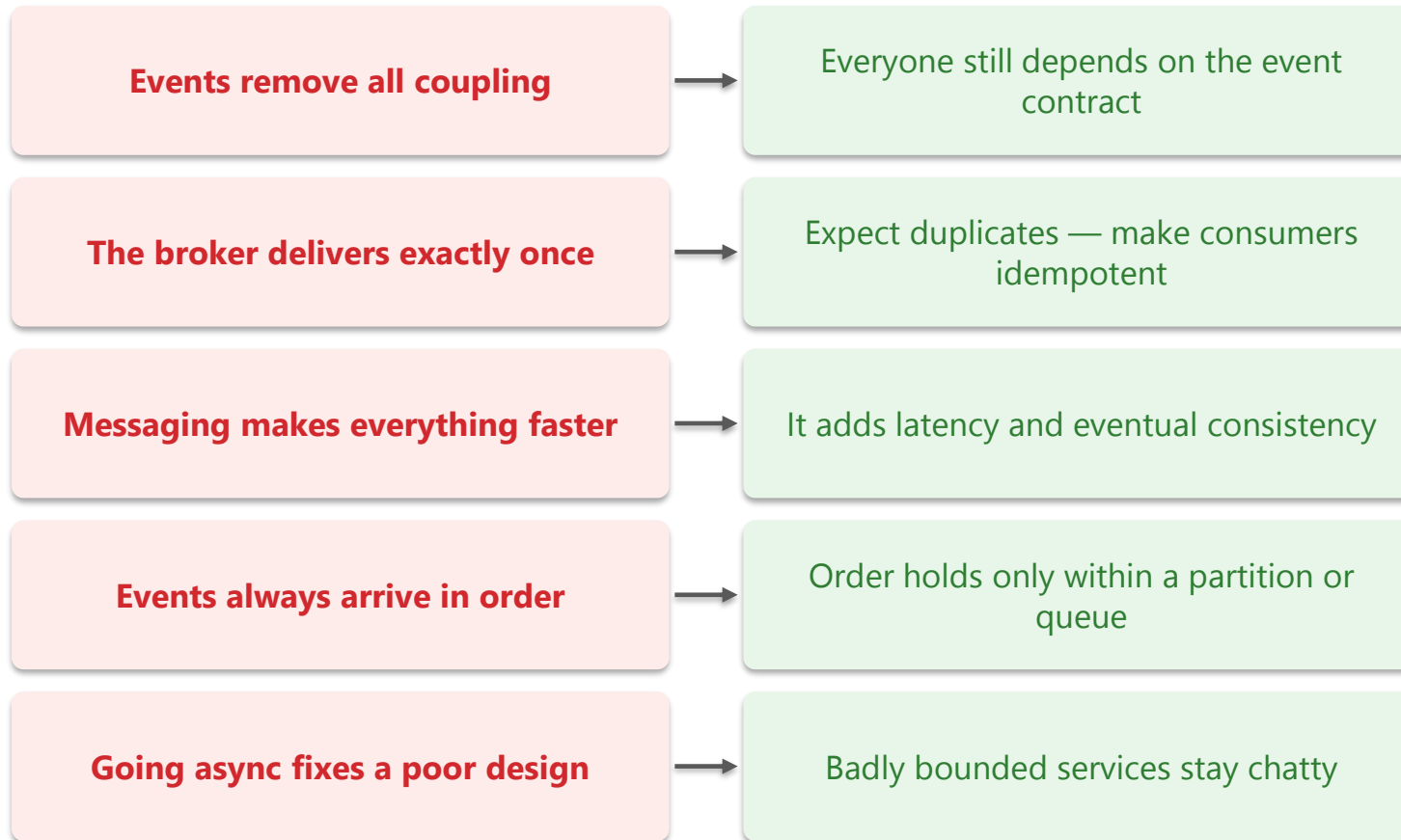
KEY TAKEAWAY Events let services collaborate without being simultaneously available — if you design for reliability.

EVENT-DRIVEN MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs and habits that make messaging systems fragile.

MISCONCEPTION

REALITY



ANTI-PATTERNS TO AVOID

- ✗ Publishing before the transaction commits
- ✗ Vague events like DataChanged
- ✗ Ignoring duplicate deliveries
- ✗ Retrying poison messages forever
- ✗ Card numbers or tokens in payloads
- ✗ No lag or dead-letter monitoring

REMEMBER Events decouple timing, not responsibility — every message needs an owner.

KEY TAKEAWAY Messaging only helps when contracts, duplicates, failures and monitoring are designed in from the start.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 Synchronous vs. asynchronous — the main difference?

2 When is a REST call appropriate between services?

3 Why are long chains of synchronous calls risky?

4 What is an event?

5 Why name events in the past tense?

6 What role does a message broker play?

7 What is a dead-letter queue for?

8 Why must event consumers be idempotent?

9 What is consumer lag in Kafka?

10 Why should services avoid sharing databases?

MODULE 4 SUMMARY AND WHAT'S NEXT

From synchronous calls to a reliable event-driven workflow.

YOU CAN NOW

- ✓ Choose synchronous or asynchronous for each interaction
- ✓ Explain brokers, queues, topics and consumer groups
- ✓ Design events, producers and consumers that stay decoupled
- ✓ Pick choreography or orchestration for a workflow
- ✓ Build idempotent consumers with retries and dead letters

NEXT: MODULE 5 — MICROSERVICES AND DATA

- 1 How does each service own and persist its data?
- 2 How do we keep a workflow correct across databases?
- 3 When does caching help — and when does it hurt?

BRING ALONG Your Lab 2 Transaction Service and its TransactionSubmitted flow.

KEY TAKEAWAY Module 5 looks inside each service's data — ownership, transactions and caching.